

BMIT3173 INTEGRATIVE PROGRAMMING

ASSIGNMENT 202605

MediCare Connect

PATIENT RECORD MODULE

Student Name	Heng Chin Yang
Student ID	25WMR09726
Programme	Bachelor of Information Technology (Honours) in Information Security
Tutorial Group	RIS3G5
System Title	MediCare Connect
Chosen SDG	SDG 3: Good Health and Well-Being
Individual Module	Patient Record
Submission Date	6 September 2026

Individual Technical Report

Student Declaration and AI Usage Disclosure

Plagiarism statement

I understand that plagiarism is presenting another person's work, ideas, code, or wording as my own. I will ensure that the submitted report accurately describes the implementation I can explain and demonstrate, and that all external sources and permitted AI assistance are acknowledged.

Student signature

Date

AI usage disclosure

Generative AI assistance was used during development and report preparation. The assistance included concept explanations, code review, debugging support, test planning, and drafting/formatting this report. I reviewed the suggestions, adapted them to the Patient Record module, executed the automated tests, and remain responsible for the accuracy of the submitted work and for explaining it during the demonstration.

Tool	Purpose	How the output was handled
OpenAI ChatGPT / Codex	Explanations, code review, debugging, integration-test planning, and report drafting.	Suggestions were reviewed against the actual PHP project and verified through local automated tests.

Student action before submission: confirm the student ID, sign and date this page, and adjust the disclosure if any additional AI tools were used.

Table of Contents

Student Declaration and AI Usage Disclosure	2
Plagiarism statement	2
AI usage disclosure	2
Table of Contents	3
1. Introduction to the System	5
1.1 System overview	5
1.2 Sustainable Development Goal	5
1.3 Scope and target users	5
1.4 Technology and architecture	5
2. Module Description	6
2.1 Main functions	6
2.2 MVC request lifecycle	6
2.3 Creating a record with medicines	6
2.4 Interface evidence	7
2.5 Robustness and maintainability	7
2.6 Automated verification result	7
3. Entity Classes and ORM	8
3.1 Entity class diagram	8
3.2 Entity responsibilities	8
3.3 Eloquent ORM implementation	8
3.4 Encryption through an ORM attribute cast	9
4. Design Pattern: Decorator	10
4.1 Description of the pattern	10
4.2 Implementation class diagram	10
4.3 Mapping the diagram to PHP classes	10
4.4 Runtime implementation	11
4.5 Why Decorator is suitable	11
4.6 Limitation and mitigation	11
5. Software Security	12
5.1 Threat 1: SQL injection	12
5.2 Threat 2: data breach through unencrypted stored data	13
5.3 Additional defence-in-depth controls	13
6. Web Services	14

6.1 Technology choice	14
6.2 Service exposure: Patient Record summary	14
6.3 Service consumption: Pharmacy	15
6.4 Pharmacy availability request and response	16
6.5 Multiple-medicine approved dispensing request	16
6.6 Cross-module data ownership	17
6.7 REST/XML implementation extract	17
6.8 Automated integration and security tests	17
6.9 Reflection and Independent Learning	18

7. References 19

7.1 Source-code evidence index	19
--	----

The report follows the section order required by the supplied assignment report template and addresses the highest-band marking criteria for functionality, MVC/ORM, design pattern, secure coding, web services, and independent learning.

1. Introduction to the System

1.1 System overview

MediCare Connect is a modular healthcare information system developed by a team of four students. Its four modules are User Management, Appointment, Pharmacy, and Patient Record. Each module is deployed as a separate PHP folder under the XAMPP htdocs directory and owns its own database. Cross-module information is exchanged through REST endpoints carrying XML messages, instead of reading another module's database directly.

My individual responsibility is the Patient Record module. It stores a patient's clinical record history, links each record to a condition and responsible doctor, records access activity, and stores only an external prescription reference for Pharmacy-owned prescription details. A patient may view only their own records. A doctor may view and maintain records only for patients assigned to that doctor, while an administrator has authorised maintenance access.

1.2 Sustainable Development Goal

The chosen goal is United Nations SDG 3: Good Health and Well-Being. The system supports continuity of care by making authorised clinical history available to the right user, reducing fragmented information, and linking medical-record creation with Pharmacy stock validation. These functions do not replace clinical judgement; they support safer information handling and more coordinated service delivery.

1.3 Scope and target users

- Patients review their own medical history and linked prescription outcome.
- Doctors select an assigned patient, create or update records, select one or more medicines, and receive immediate stock/integration feedback.
- Administrators and doctors maintain the reusable condition catalogue according to role permissions.
- Other modules consume the Patient Record XML service using the agreed request/response fields.

1.4 Technology and architecture

Layer / concern	Implementation
Language and runtime	PHP 8.2 on XAMPP/Apache with MySQL
MVC	index.php routes to controllers; controllers coordinate models/services and load PHP views
ORM	Illuminate Database / Eloquent 12, used as standalone Composer packages
Design pattern	Decorator for confidentiality, access context, and severe-condition alert behaviour
Integration	REST over HTTP with XML payloads, XSD validation, requestID, timestamp, status, and API key
Security	Bound ORM queries, AES-256-GCM encryption, access control, CSRF, hardened sessions, and output escaping

2. Module Description

2.1 Main functions

Function	User behaviour and rule	Main implementation path
Authentication boundary	Uses the shared role, username, user_id, and patient_id session contract. Unauthenticated requests receive HTTP 401.	Shared/SessionSecurity.php; index.php
Patient history	A patient sees only records whose patient_id matches the authenticated patient_id.	PatientRecordController::index(); View/patient_record/index.php
Doctor patient list	A doctor sees existing responsible patients and appointment assignments returned by REST, solving the first-record problem.	doctorPatients(); AppointmentServiceClient.php
Create record	The doctor ID is taken from the session. Patient and doctor identifiers are not typed into the form. Condition, severity, date, remark, and medicines are validated.	PatientRecordController::store()
Pharmacy workflow	The user selects one or many catalogue medicines. Stock is checked again on the server before an approved dispensing request is created.	PatientRecordCreationService.php; PharmacyServiceClient.php
View record	Displays the record, decorator effects, access time, severe-condition alert, and linked Pharmacy prescription details.	PatientRecordController::show(); show.php
Audit trail	VIEW, CREATE, and UPDATE events are saved with accessor ID, name, role, type, and time.	PatientRecordAccessLog.php
Condition maintenance	Doctors/admins add and modify the medical-condition catalogue; patients are denied.	ConditionController.php

2.2 MVC request lifecycle

The front controller starts the secure session, rejects unauthenticated users, loads the ORM and controllers, checks CSRF for every modifying POST, and dispatches the requested action. The controller performs role and ownership checks, calls the model or integration service, and selects the view. Views do not build SQL queries.

```
// index.php (simplified)
SessionSecurity::start();
if (!SessionSecurity::isAuthenticated()) { /* HTTP 401 */ }
require 'Shared/orm.php';
$controller = new PatientRecordController();
switch ($action) {
  case 'index': $controller->index(); break;
  case 'store': $controller->store(); break;
  case 'show': $controller->show($id); break;
}
```

Code Extract 1. Front-controller routing and MVC separation (index.php).

2.3 Creating a record with medicines

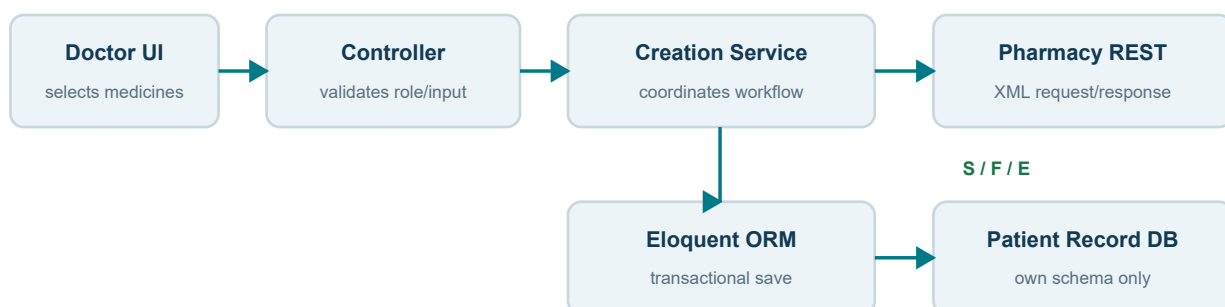


Figure 1. Patient Record creation flow and REST boundary.

The form catalogue improves usability, but it is not trusted as a security boundary. The server validates every SKU and quantity and then calls Pharmacy for real-time availability. If any item is unavailable, the local record is not saved. If Pharmacy succeeds but the local database fails, the generated prescription reference is logged for reconciliation and the user receives a controlled service error.

2.4 Interface evidence

Patient Record Details

Record PR010

CONFIDENTIAL MEDICAL INFORMATION

Patient information

PATIENT NAME	PATIENT ID
Aisyah Rahman	PA002
RECORD DATE	SEVERITY
2026-09-04	Moderate

Clinical information

CONDITION	DOCTOR ID	PRESCRIPTION REFERENCE
Asthma	DC001	RX-3A8D6F91

Remark: Asthma symptoms reviewed; follow prescription instructions.

Accessed by: Doctor DC001 Access time: 2026-09-04 14:20:31

Figure 2. Final Patient Record detail-view layout implemented by View/patient_record/show.php.

The final interface uses the Pharmacy module's shared stylesheet and favicon to give the team system a consistent appearance. Decorator effects are integrated into the detail card as a confidentiality label, access information, and a conditional emergency alert rather than being displayed as duplicate debug output.

2.5 Robustness and maintainability

- Pagination limits record-history pages to ten records and clamps invalid page numbers.
- Transactions protect local record and prescription-reference creation from partial local writes.
- HTTP errors use suitable status codes, including 401, 403, 404, 405, 419, 422, 502, and 503.
- External-service timeouts are five seconds and failures are converted to safe user-facing messages.
- Module-specific database accounts enforce that Patient Record cannot query Pharmacy or User Management tables.

2.6 Automated verification result

On 4 September 2026, the three command-line test suites were executed with C:\xampp\php\php.exe. All application-security, Pharmacy integration, and Patient Record XML service checks passed. The detailed cases are reported in Section 6.8.

3. Entity Classes and ORM

3.1 Entity class diagram

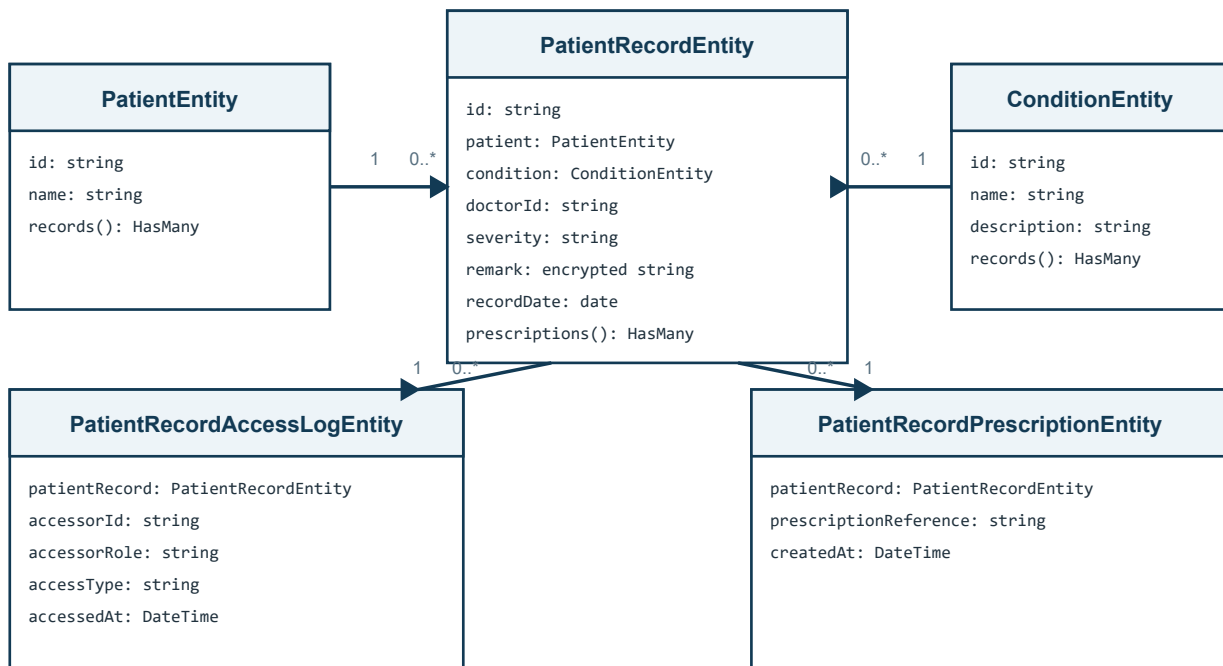


Figure 3. Patient Record Eloquent entity classes and object relationships.

The diagram is a class-oriented view, not an entity-relationship diagram. Relationships are represented in PHP as object-returning Eloquent methods such as `patient()`, `condition()`, `prescriptions()`, and `patientRecord()`. The database still uses foreign-key columns internally, but controllers work through entity objects and relations.

3.2 Entity responsibilities

Entity class	Table	Responsibility and relations
PatientEntity	patients	Patient identity cached for this module; has many PatientRecordEntity objects.
ConditionEntity	conditions	Reusable condition name/description; has many PatientRecordEntity objects.
PatientRecordEntity	patient_records	Clinical record aggregate; belongs to PatientEntity and ConditionEntity; has prescriptions.
PatientRecordPrescriptionEntity	patient_record_prescriptions	Stores only the external Pharmacy prescription reference; belongs to PatientRecordEntity.
PatientRecordAccessLogEntity	patient_record_access_logs	Persistent VIEW/CREATE/UPDATE audit events; belongs to PatientRecordEntity.

3.3 Eloquent ORM implementation

Eloquent is installed through Composer as illuminate/database version 12. Models use string primary keys because the agreed identifiers are PA001, DC001, and PR001 formats. Eager loading prevents repeated patient/condition lookups while the ORM's query builder binds values instead of concatenating them into SQL.

```
// Model/PatientRecord.php
$records = PatientRecordEntity::query()
    ->with(['patient', 'condition', 'prescriptions'])
    ->where('patient_id', $patientId)
    ->where('doctor_id', $doctorId)
    ->orderByDesc('record_date')
    ->get();
```

Code Extract 2. Eloquent relationships and parameter-bound query construction.

3.4 Encryption through an ORM attribute cast

PatientRecordEntity defines a custom Eloquent Attribute for remark. The set transformation encrypts before the value reaches MySQL; the get transformation decrypts only after an authorised query. Therefore controllers and views work with normal text while the database stores authenticated ciphertext.

```
protected function remark(): Attribute
{
    return Attribute::make(
        get: fn (?string $v) => SensitiveDataCipher::instance()->decrypt($v ?? ''),
        set: fn (?string $v) => SensitiveDataCipher::instance()->encrypt($v ?? '')
    );
}
```

Code Extract 3. Transparent encryption/decryption at the ORM boundary.

4. Design Pattern: Decorator

4.1 Description of the pattern

Decorator is a structural design pattern that adds responsibilities to an object by wrapping it in another object with the same interface. Each wrapper delegates to the wrapped component and adds behaviour before or after the delegation. The wrappers can be stacked in different combinations without creating subclasses for every possible combination of confidentiality, auditing, and emergency warning.

4.2 Implementation class diagram

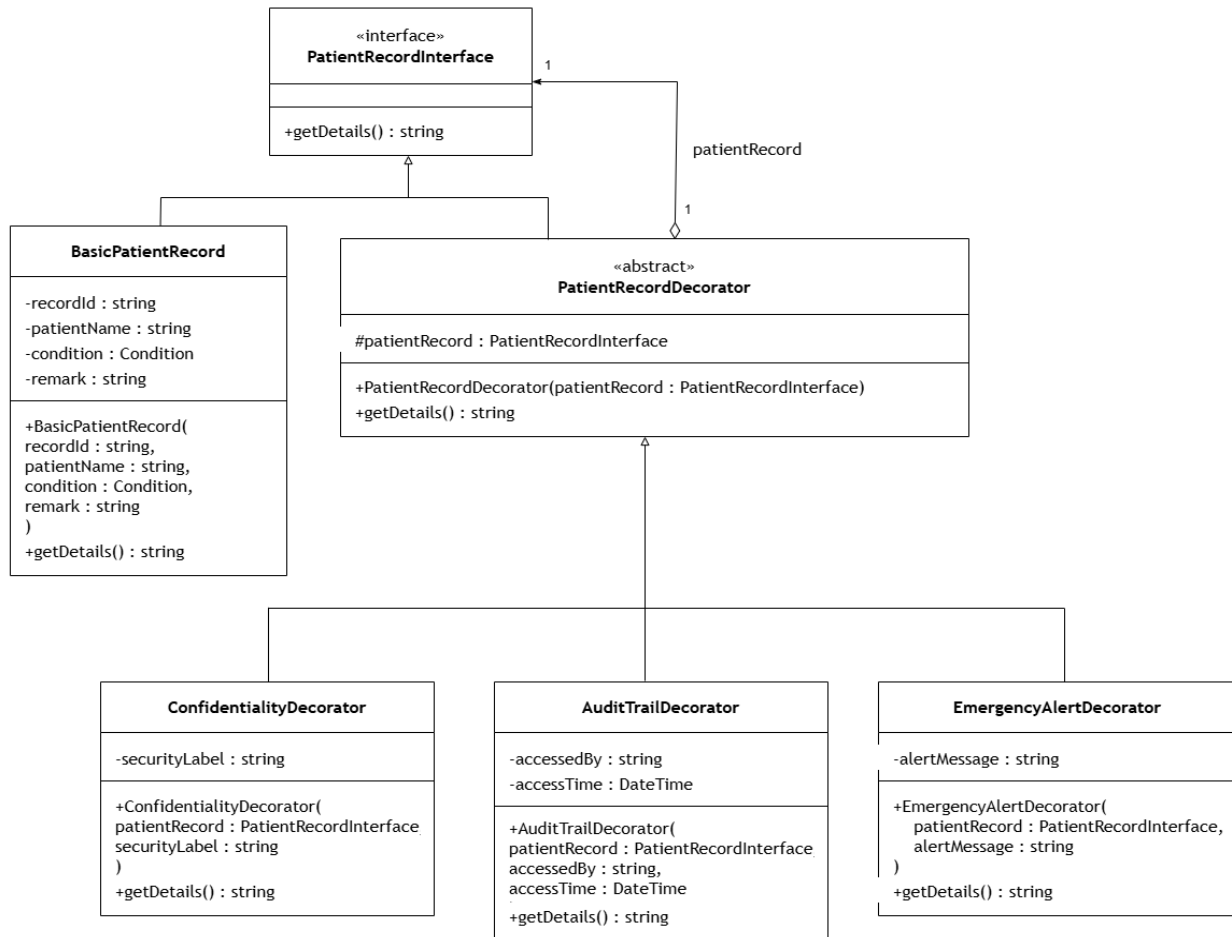


Figure 4. Decorator design-pattern class diagram used by the Patient Record module.

4.3 Mapping the diagram to PHP classes

Pattern role	Project class	Behaviour
Component	PatientRecordInterface	Defines <code>getDetails(): string</code> for every base or decorated record.
Concrete component	BasicPatientRecord	Builds the normal record ID, patient, condition, and remark representation.
Base decorator	PatientRecordDecorator	Stores a protected PatientRecordInterface reference and delegates <code>getDetails()</code> .
Concrete decorator	ConfidentialityDecorator	Adds a security classification label.
Concrete decorator	AuditTrailDecorator	Adds the accessor and access time to the representation.
Concrete decorator	EmergencyAlertDecorator	Adds a warning only when severity is Severe.

4.4 Runtime implementation

The controller starts with `BasicPatientRecord`, then wraps it with confidentiality and audit decorators. The `EmergencyAlertDecorator` is added conditionally. Since every wrapper implements the same interface, the view receives one final `PatientRecordInterface` representation regardless of the number of decorators. Separately, the audit event is persisted in `patient_record_access_logs` so that the user-visible decoration is supported by durable evidence rather than being only display text.

```
$patientRecord = new BasicPatientRecord($id, $name, $condition, $remark);
$patientRecord = new ConfidentialityDecorator($patientRecord, $securityLabel);
$patientRecord = new AuditTrailDecorator($patientRecord, $accessedBy, $accessTime);
if ($severity === 'Severe') {
    $patientRecord = new EmergencyAlertDecorator($patientRecord, $message);
}
$decoratedDetails = $patientRecord->getDetails();
```

Code Extract 4. Decorators composed at runtime in `PatientRecordController::show()`.

4.5 Why Decorator is suitable

- It solves a real cross-cutting presentation problem: a record may require confidentiality, auditing context, and an emergency alert at the same time.
- Open/closed principle: a future `MaskedPatientNameDecorator` or `PrintingWatermarkDecorator` can be added without modifying `BasicPatientRecord`.
- Single responsibility: each concrete decorator has one clear addition, making testing and explanation easier.
- Runtime flexibility: the severe alert is included only for Severe records while the other wrappers remain unchanged.

Inheritance alone would require many combined subclasses, for example `ConfidentialAuditRecord` and `ConfidentialAuditEmergencyRecord`. Decorator avoids this class explosion. A simple helper function would also work for formatting, but it would not demonstrate substitutable objects or allow independent composition and testing.

4.6 Limitation and mitigation

Decorator order affects the exact output order, and many small wrapper objects can make debugging harder. The module mitigates this by composing the decorators in one controller method, using descriptive class names, and testing all three decorators independently. `ConfidentialityDecorator` is not treated as encryption or access control; those security controls are implemented separately.

5. Software Security

5.1 Threat 1: SQL injection

SQL injection occurs when untrusted input becomes part of the SQL command structure. For example, if `patient_id` were concatenated into a `WHERE` clause, an attacker could add SQL syntax to bypass ownership filtering, disclose another patient's records, change data, or damage tables. Healthcare records make the confidentiality and integrity impact especially serious.

Secure coding practice: strongly typed parameterised ORM queries

All application database operations use Eloquent's query builder or entity save/create methods. Values passed to `where()`, `find()`, `create()`, and `update()` are sent through PDO bindings instead of string concatenation. In addition, identifiers are allow-listed using formats such as `^PA[0-9]{3,8}$`, quantities are validated as integers, severity uses a fixed list, and record dates must round-trip through `DateTimeImmutable`. Parameterisation is the main SQL injection control; validation provides defence in depth.

```
public function findById(string $recordId): ?array
{
    $record = PatientRecordEntity::query()
        ->with(['patient', 'condition', 'prescriptions'])
        ->find($recordId); // bound string primary-key value
    return $record === null ? null : $this->toArray($record);
}
```

Code Extract 5. No user-controlled value is concatenated into SQL.

How the control can be demonstrated

- Request a record with an invalid identifier such as `PR001' OR '1'='1`; the ID format check returns not found and no SQL syntax is executed.
- Use a valid ID and show that only that bound value is queried.
- Show `Model/PatientRecord.php` and the Eloquent entity methods to the examiner.
- Explain that escaping output prevents XSS, but it is not the SQL injection control.

5.2 Threat 2: data breach through unencrypted stored data

A database backup, copied data directory, compromised database account, or administrator mistake can expose stored medical remarks even if the web screens are protected. Plaintext clinical notes reveal sensitive health information immediately. Access control alone does not protect against an attacker who obtains only the database.

Secure coding practice: AES-256-GCM authenticated encryption

SensitiveDataCipher encrypts clinical remarks with AES-256-GCM. A fresh 12-byte random IV is generated for each value, and the 16-byte authentication tag detects modification. The stored value begins with enc:v1: so encrypted rows can be identified and a later key-version migration is possible. Additional authenticated data binds the ciphertext to the Patient Record remark purpose.

```
$iv = random_bytes(12);
$ciphertext = openssl_encrypt(
    $plaintext, 'aes-256-gcm', $key, OPENSSL_RAW_DATA,
    $iv, $tag, 'medicare-connect:patient-record-remark:v1', 16
);
return 'enc:v1:' . base64_encode($iv . $tag . $ciphertext);
```

Code Extract 6. Authenticated encryption for sensitive clinical remarks.

The 32-byte encryption key is read from a file outside httdocs and is not stored in the database or source repository. If the key is missing, malformed, or the ciphertext authentication fails, the operation fails closed. This supports key/data separation recommended for protecting stored data.

How the control can be demonstrated

- Create a record containing a recognisable phrase, then query patient_records.remark in phpMyAdmin; only an enc:v1: Base64 payload should be visible.
- Open the same record through the authorised application; the ORM accessor decrypts it for display.
- Run tests/application_security_test.php to show encryption, correct decryption, and tamper rejection all pass.

5.3 Additional defence-in-depth controls

Control	Implementation and purpose
Role and ownership checks	Patients only access their patient_id; doctors require responsibility; admins have explicit privileged access.
CSRF tokens	Every modifying browser POST must contain a 256-bit random token verified with hash_equals().
Hardened sessions	Strict mode, cookie-only sessions, HttpOnly, SameSite=Lax, Secure on HTTPS, and session-ID regeneration.
Output encoding	htmlspecialchars() is used by views to stop stored or reflected HTML/script execution.
API authentication	REST services require X-API-Key values loaded from files outside the web root.
XML hardening	DTD/entity declarations are rejected; LIBXML_NONET prevents network entity resolution; XSD validates structure.
Least-privilege database	patient_record_app has access only to medicare_connect2; other modules have separate credentials.
Audit logging	VIEW, CREATE, and UPDATE events are written to a related log table with identity, role, action, and time.

6. Web Services

6.1 Technology choice

The module uses REST-style HTTP endpoints with XML payloads. REST was selected because the teammates' PHP modules already expose HTTP routes and do not require SOAP envelopes or WSDL tooling. XML remains the agreed data format, and XSD supplies strict machine-verifiable message contracts. Each module owns its database, so REST is the only supported way to obtain cross-module information.

6.2 Service exposure: Patient Record summary

IFA Item	Value
Protocol	REST / HTTP POST with application/xml
Function	Retrieve authorised Patient Record summary data by patient ID
Source module	Patient Record
Target module	Appointment, Pharmacy, or another approved MediCare Connect module
URL	http://localhost/project/api/patient-record-summary.php
Authentication	X-API-Key request header
Implementation	api/patient-record-summary.php; WebService/PatientRecordXmlService.php

Request parameters

Field	Type	M/O	Description	Format
requestID	String	M	Unique correlation ID generated by the caller	1-64 letters, digits, _ or -
timeStamp	String	M	Time the request was generated	YYYY-MM-DD HH:MM:SS; within 5 minutes
patientID	String	M	Patient whose summary is required	PA followed by 3-8 digits

```
<?xml version="1.0" encoding="UTF-8"?>
<patientRecordRequest>
  <requestID>REQ-DEMO-001</requestID>
  <timeStamp>2026-09-04 14:00:00</timeStamp>
  <patientID>PA002</patientID>
</patientRecordRequest>
```

XML Example 1. Patient Record service request.

Response parameters

Field	Type	M/O	Description
requestID	String	M	Echoes the request correlation ID.
status	String	M	S = success, F = business failure/not found, E = malformed/technical/authentication error.
timeStamp	String	M	Server response time in YYYY-MM-DD HH:MM:SS.
message	String	M	Human-readable result without internal exception details.
patient	Object	O	patientID and patientName when data is found.
records	List	O	recordID, doctorID, condition, severity, and recordDate. Sensitive remark is intentionally excluded.

```
<?xml version="1.0" encoding="UTF-8"?>
<patientRecordResponse>
  <requestID>REQ-DEMO-001</requestID><status>S</status>
  <timeStamp>2026-09-04 14:00:01</timeStamp>
  <message>Patient Record summary retrieved successfully.</message>
  <patient><patientID>PA002</patientID><patientName>Aisyah Rahman</patientName></patient>
  <records><record><recordID>PR010</recordID><doctorID>DC001</doctorID>
    <condition>Asthma</condition><severity>Moderate</severity>
    <recordDate>2026-09-04</recordDate></record></records>
</patientRecordResponse>
```

XML Example 2. Successful Patient Record summary response.

Exposure safeguards

- POST is mandatory; other methods receive 405 and an XML error response.
- The API key is compared securely and failed authentication receives 401/E.
- Request size is limited to 65,536 bytes and stale timestamps are rejected.
- Request and response must validate against patient_record_request.xsd and patient_record_response.xsd.
- Clinical remarks are excluded from the exposed summary to minimise sensitive data disclosure.

6.3 Service consumption: Pharmacy

IFA Item	Value
Protocol	REST / HTTP POST with namespaced XML
Source module	Patient Record
Target module	Pharmacy
URL	http://localhost/MediCareConnect-Pharmacy-Lawliet/MediCareConnect-Pharmacy-Lawliet/api/pharmacy
Operations	getMedicineCatalog; getMedicineAvailability; createApprovedDispensingRequest; getPrescription
Authentication	X-API-Key loaded from medicare-connect-secrets/pharmacy-service.key
Implementation	Service/PharmacyServiceClient.php and Service/xsd/pharmacy_*.xsd

6.4 Pharmacy availability request and response

Before a Patient Record is saved, each selected medicine is checked with the requested quantity. The response must have HTTP 200, status S, the same requestID, the same SKU and quantity, and available equal to true. The local form's maximum is a usability feature; the server-side Pharmacy response is the authoritative stock decision.

```
<m:getMedicineAvailability xmlns:m="urn:medicare:patient-record">
  <m:requestID>REQ-A1B2C3D4</m:requestID>
  <m:timestamp>2026-09-04 14:10:00</m:timestamp>
  <m:sku>MED-SALB-100</m:sku><m:quantity>2</m:quantity>
</m:getMedicineAvailability>

<m:getMedicineAvailabilityResponse xmlns:m="urn:medicare:patient-record">
  <m:status>S</m:status><m:timestamp>2026-09-04 14:10:01</m:timestamp>
  <m:message>Availability checked.</m:message><m:requestID>REQ-A1B2C3D4</m:requestID>
  <m:sku>MED-SALB-100</m:sku><m:requestedQuantity>2</m:requestedQuantity>
  <m:available>true</m:available><m:availableQuantity>7</m:availableQuantity>
</m:getMedicineAvailabilityResponse>
```

XML Example 3. Correlated medicine-availability request and response.

6.5 Multiple-medicine approved dispensing request

The request contains a generated prescriptionReference, patient and prescriber external identifiers, and one or more item elements. The Patient Record service accepts success only when Pharmacy returns status S, the matching requestID/reference, and dispensingStatus approved. Pharmacy owns the medicine and dispensing data; the local database stores only the returned reference.

```
<m:createApprovedDispensingRequest xmlns:m="urn:medicare:patient-record">
  <m:requestID>REQ-9F21</m:requestID><m:timestamp>2026-09-04 14:12:00</m:timestamp>
  <m:prescriptionReference>RX-3A8D6F91</m:prescriptionReference>
  <m:patientExternalId>PA002</m:patientExternalId><m:patientName>Aisyah Rahman</m:patientName>
  <m:prescriberExternalId>DC001</m:prescriberExternalId><m:prescriberName>Doctor DC001</m:
prescriberName>
  <m:items>
    <m:item><m:sku>MED-PARA-500</m:sku><m:quantity>10</m:quantity>
    <m:instructions>Take after food.</m:instructions></m:item>
    <m:item><m:sku>MED-SALB-100</m:sku><m:quantity>1</m:quantity>
    <m:instructions>Use when required.</m:instructions></m:item>
  </m:items>
</m:createApprovedDispensingRequest>
```

XML Example 4. Multiple medicines in one approved dispensing request.

6.6 Cross-module data ownership

Module	Database it owns	Patient Record obtains external data by
Patient Record	medicare_connect2	Direct Eloquent access using patient_record_app
Pharmacy	medicare_pharmacy	Pharmacy REST/XML endpoint only
User Management / Appointment assignment	clinic_db / teammate-owned schema	Shared authenticated session and doctor-patient assignment REST/XML endpoint

This separation proves that the service integration is necessary: the Patient Record database account has no permission to select from Pharmacy or User Management tables. If an external module is unavailable, the Patient Record code handles the failure instead of silently bypassing the service boundary.

6.7 REST/XML implementation extract

```
$client = curl_init($serviceUrl);
curl_setopt_array($client, [
    CURLOPT_POST => true, CURLOPT_POSTFIELDS => $requestXml,
    CURLOPT_RETURNTRANSFER => true, CURLOPT_CONNECTTIMEOUT => 5, CURLOPT_TIMEOUT => 5,
    CURLOPT_HTTPHEADER => [
        'Content-Type: application/xml', 'Accept: application/xml', 'X-API-Key: ' . $apiKey
    ],
]);
$body = curl_exec($client);
```

Code Extract 7. REST transport used by PharmacyServiceClient.

After transport, the client rejects oversized, unsafe, malformed, unexpected-root, or XSD-invalid XML. It then checks mandatory fields, status, requestID correlation, item identifiers, and operation-specific success fields.

6.8 Automated integration and security tests

Suite	Verified scenarios	Result
Application security	Missing identity; session contract; doctor session; account switching; patient identity cleanup; valid/invalid CSRF; encrypt/decrypt/tamper; three decorators	13/13 PASS
Pharmacy REST/XML	Catalogue; available/unavailable; invalid SKU; multiple medicines; approved creation; duplicate/idempotent reference; unavailable service; malformed XML; wrong API key; Pharmacy success plus local failure	11/11 PASS
Patient Record XML exposure	HTTP 200/S; no clinical remark disclosure; invalid XML; missing key; unsupported method; stale timestamp; unknown patient	8/8 PASS

The Pharmacy tests use controlled transport doubles, so success and failure responses can be reproduced without changing live stock. The XML smoke test exercises the actual service class and endpoint wrapper. These tests show normal, boundary, authentication, malformed-input, dependency-failure, idempotency, and partial-failure behaviour.

6.9 Reflection and Independent Learning

6.9.1 Knowledge developed

This module required more than implementing CRUD forms. I learned how to use Eloquent outside a full Laravel application, model object relationships, and apply an authenticated-encryption transformation at the entity boundary. I also learned that a design-pattern label is not itself a security feature: the ConfidentialityDecorator communicates classification, while access control and encryption provide actual protection.

The integration work showed why database sharing creates tight coupling. Restricting each database account forced the Patient Record module to use explicit contracts. Request IDs detect mismatched responses, timestamps limit replay windows, XSD catches structural mistakes, and deterministic error status values help other modules respond correctly. The Pharmacy-success/local-failure case also demonstrated that distributed operations cannot rely on a single MySQL transaction; they require reconciliation evidence.

6.9.2 Problems solved and decisions

- The original vendor autoloader was incomplete. Composer dependencies were restored so Eloquent could load reliably.
- Patients were initially able to reach edit behaviour. Role and record-ownership rules now ensure patients are read-only.
- An edit originally risked changing the patient owner. Update now reuses the existing record's patient_id and never accepts ownership from the form.
- Doctor lists originally depended only on existing records, preventing the first record. REST assignment data now provides the authorised patient before a record exists.
- Medicine quantity was initially only a browser constraint. Server-side Pharmacy availability is now authoritative and is rechecked immediately before saving.
- Decorator output originally appeared as a second debug panel. Its effects are now integrated into the normal detail view, and audit evidence is stored in the database.

6.9.3 Current limitations and future work

The development environment uses HTTP localhost, so the Secure session-cookie flag becomes effective only when the final deployment uses HTTPS. A production system should use managed key storage with rotation, structured central logging, rate limiting on APIs, and a formal compensating action in Pharmacy rather than only logging a reconciliation reference. End-to-end tests across all teammates' deployed modules should also run in a shared integration environment before submission.

The supplied team class diagram includes earlier Appointment fields that are no longer stored in the Patient Record database. This report therefore presents the implemented Patient Record entity model, while appointment assignment remains external service data. This keeps the documentation consistent with the submitted source code.

7. References

- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine). <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Laravel. (2026). Eloquent ORM: Getting started (Laravel 12.x). <https://laravel.com/framework/docs/12.x/eloquent>
- OpenAI. (2026). ChatGPT and Codex [Large language model]. <https://openai.com/codex/>
- OWASP Foundation. (2026a). Cryptographic storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- OWASP Foundation. (2026b). SQL injection prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- PHP Documentation Group. (2026). Prepared statements and stored procedures. <https://www.php.net/manual/en/pdo.prepared-statements.php>
- United Nations. (2026). Goal 3: Ensure healthy lives and promote well-being for all at all ages. <https://sdgs.un.org/goals/goal3>
- World Wide Web Consortium. (2012). W3C XML Schema Definition Language (XSD) 1.1 Part 1: Structures. <https://www.w3.org/TR/xmlschema11-1/>

7.1 Source-code evidence index

Requirement	Primary evidence in submission
MVC	index.php; Controller/; Model/; View/
ORM	Shared/orm.php; composer.json; Model/*Entity.php
Decorator	Pattern/PatientRecordSummary/; PatientRecordController::show()
SQL injection control	Eloquent model queries; typed input validation in controllers
Encrypted stored data	Shared/SensitiveDataCipher.php; PatientRecordEntity::remark()
REST exposure	api/patient-record-summary.php; WebService/PatientRecordXmlService.php
REST consumption	Service/PharmacyServiceClient.php; Service/PatientRecordCreationService.php
XML/XSD	WebService/xsd/; Service/xsd/; WebService/examples/; Service/examples/
SQL scripts	Database/schema.sql; Database/sample_data.sql; Database/migrations/
Automated tests	tests/application_security_test.php; tests/pharmacy_integration_test.php; tests/xml_service_smoke.php

Final submission reminder: the PDF should be named with the student's full name. Submit the report together with the source code and SQL create/populate scripts, and be ready to demonstrate the running module and explain the design, security controls, ORM relationships, and REST request/response flow.